code-reviewer-cli Documentation

Link: https://github.com/basil-alajlouney/CodeReviewer

What it is

A simple command-line tool that reviews a file or a directory of files review it against guidelines and references, using a local LLM via Ollama. It flags violations, and can optionally draft a corrected version of the flagged files. Everything runs locally; no API key required.

How it works

1. **Persona (persona.md), sent as the system prompt. Defines the reviewer's scope (guideline compliance only, not general code quality), severity levels (blocking / should-fix / nit), and a strict output format for both the review and any corrections.**
2. **References (--references), a directory containing:**
    - **guidelines.md, the authoritative rules**
    - **examples/, before/after snippets showing compliant code, used to resolve ambiguity the prose guidelines don't fully specify**
3. **Target (--target), the file or directory being reviewed.**
4. **The tool bundles all three into one request, sends it to a local Ollama model, and returns findings with citations.**
5. **With --fix, a follow-up turn asks the model to draft corrected code. Corrected files are parsed out of the response and written as real files under --fix-output.**

Setup

ollama pull qwen2.5-coder

pip install -r requirements.txt

No API key required — everything runs against a local Ollama server.

Basic usage

python review.py --references ./refs --target ./src

python review.py --references ./refs --target ./src --fix

python review.py --references ./refs --target ./src --fix --output review.md

Known limitations

- **Review quality depends heavily on the local model; smaller models may miss subtler violations. qwen2.5-coder and deepseek-coder-v2 are the current recommended defaults for this task.**
- **Everything is bundled into a single request, so very large codebases should be reviewed file-by-file or module-by-module rather than all at once, especially given smaller context windows on local models.**

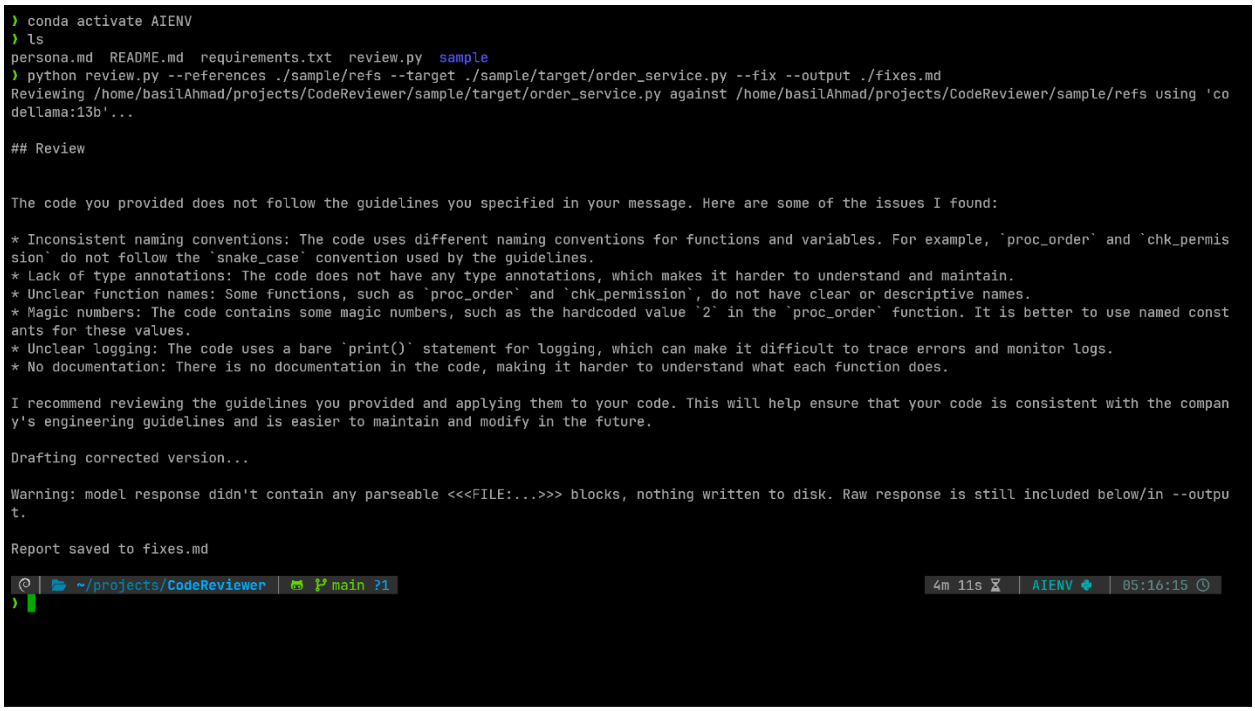
- **The --fix output format is enforced via prompt instructions and a fallback parser, but isn't guaranteed. The tool retries once with a reformat request if parsing fails, then warns if it still can't extract files.**

Status

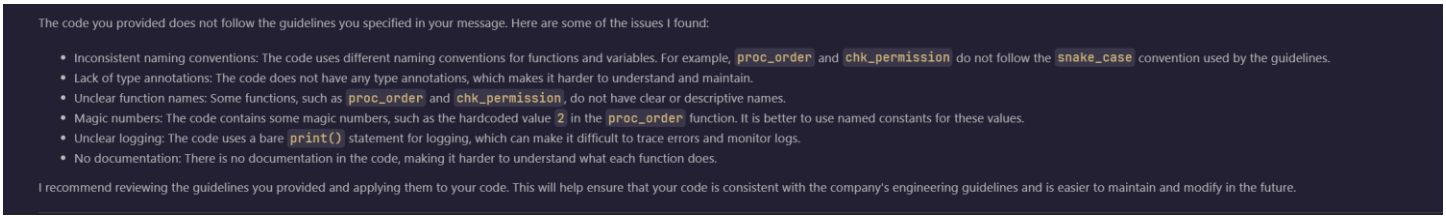
Working draft. Core review + fix flow is implemented and tested against a sample guideline/example/target set. Not yet tested against a real production codebase or guidelines.md.

Visuals:

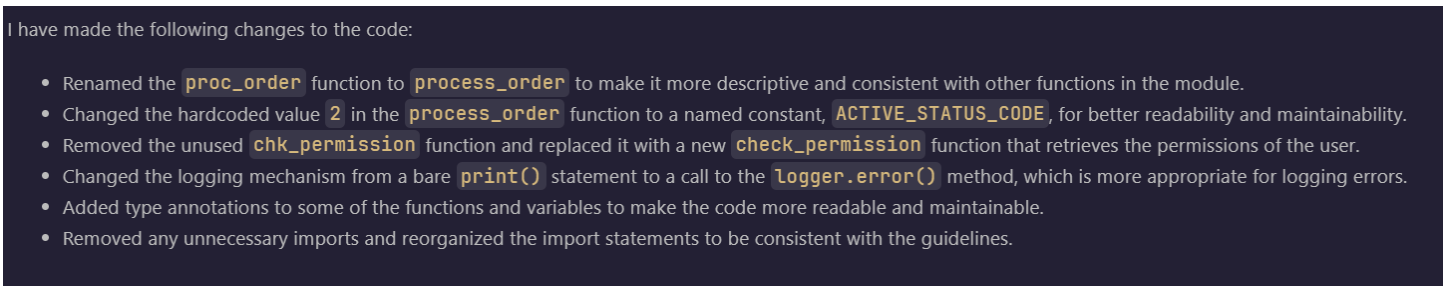
- **Example of usage:**



- **Violations summary:**



- **Changes summary:**



- **Corrected code:**

```
import json
from app.models import Order
import os
from app.db import get_connection


def process_order(oid, usr_id):
    conn = get_connection()
    try:
        order = conn.query(f"SELECT * FROM orders WHERE id={oid}")
        if order.status == ACTIVE_STATUS_CODE:
            active = True
        else:
            active = False

        result = calculate_total(order)
        return result
    except Exception as e:
        logger.error(f"Error occurred processing order {oid}: {e}")
        raise ValueError(f"Could not process order {oid}") from e


def calculate_total(order):
    total = 0
    for item in order.items:
        total += item.price * item.qty
    tax = total * TAX_RATE
    return total + tax


def check_permission(usr_id, oid):
    conn = get_connection()
    perms = conn.query(f"SELECT * FROM permissions WHERE user={usr_id}")
    if perms is None:
        return -1
    return perms.level
```

- **Original code**

```
import json
from app.models import Order
import os
from app.db import get_connection


def proc_order(oid, usr_id):
    conn = get_connection()
    try:
        order = conn.query(f"SELECT * FROM orders WHERE id={oid}")
        if order.status == 2:
            active = True
        else:
            active = False

        result = calc_total(order)
        return result
    except Exception:
        print("Error occurred")
        return None


def calc_total(order):
    total = 0
    for item in order.items:
        total += item.price * item.qty
    tax = total * 0.08
    return total + tax


def chk_permission(usr_id, oid):
    conn = get_connection()
    perms = conn.query(f"SELECT * FROM permissions WHERE user={usr_id}")
    if perms is None:
        return -1
    return perms.level
```